

CS4800: Designing Sustainable ICT Systems

Lecture 4: Sustainable AI – Part 2

September 24, 2026

Lecturer: Przemysław Pawełczak

Authors: Liwia Padowska, Przemysław Pawełczak

CS4800-EWI@tudelft.nl

What will you learn in this lecture?

One instruction, the multiply-add at the heart of every neural network, will be analyzed on the following chips:

01

Inside a CPU: tracing one instruction

See where the energy is spent.

02

Inside a GPU: one instruction, thousands of lanes

Learn why memory stalls set most of GPU's energy cost.

03

Inside a TPU & NPU: chips built for this instruction

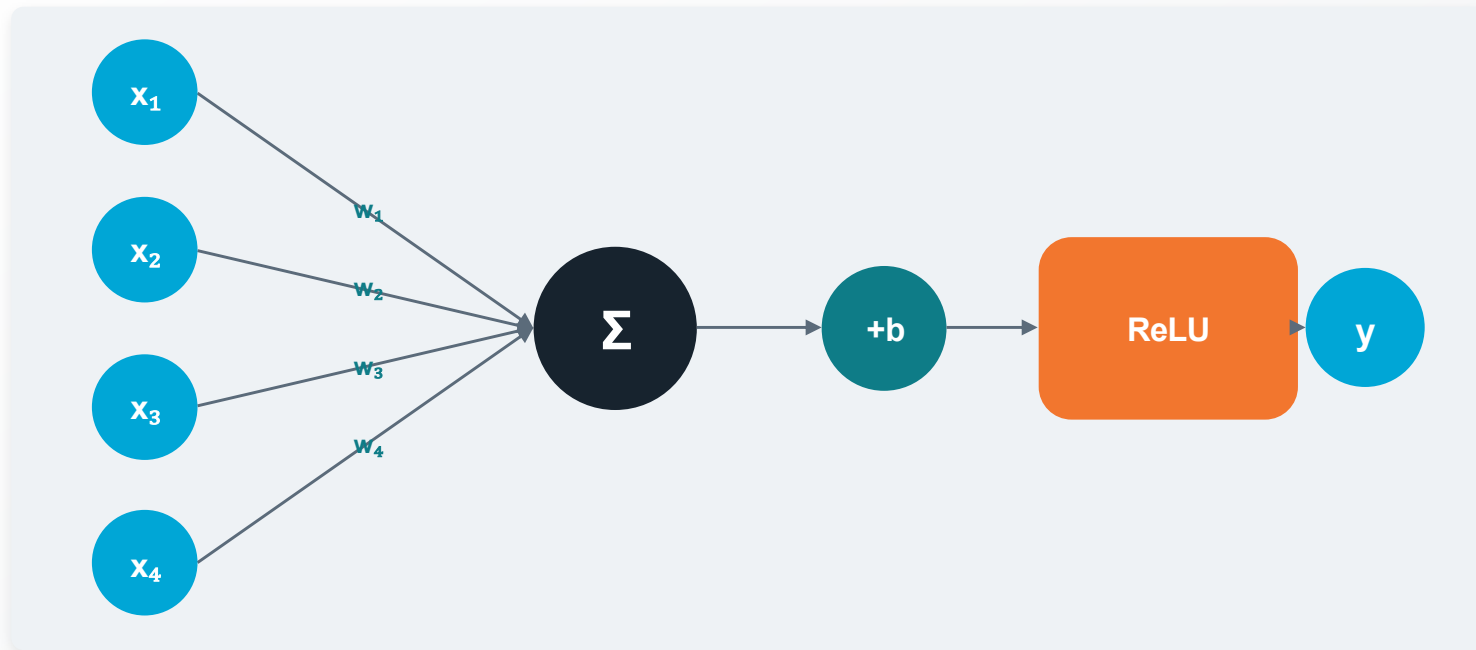
See why TPUs and NPUs trade versatility for raw matmul efficiency.

PART 2 — INSIDE THE HARDWARE

From limits to hardware: how a CPU runs your code

1.1: Our running example: one artificial neuron

From here on, we trace this computation through a CPU, a GPU, a TPU, and an NPU:



$$y = \text{ReLU}(w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b)$$

4 multiply-accumulates (MACs) $\rightarrow$ 1 bias add $\rightarrow$ 1 activation check (ReLU)

Why this operation?

Layers of neural networks are millions of copies of: multiply, accumulate, add a bias, apply a nonlinearity.

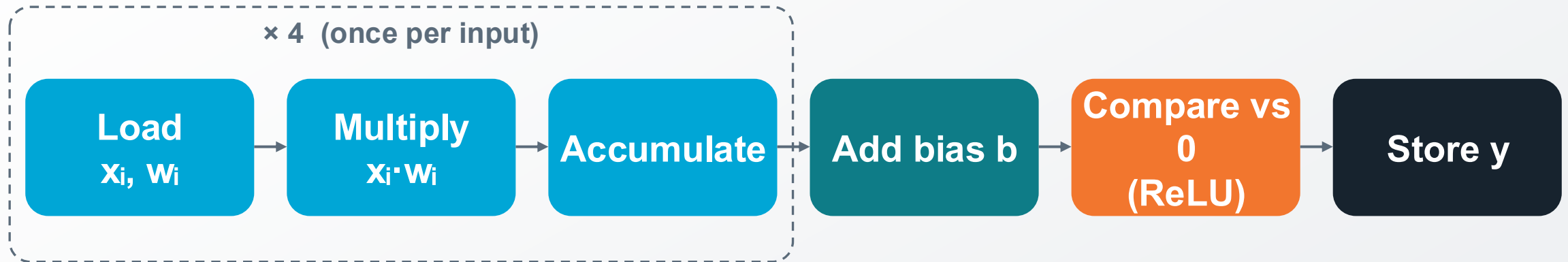
What is ReLU?

$\text{ReLU}(z) = \max(0, z)$:

pass positive signals through unchanged, zero out negative ones.

1.2: From one neuron to six operation types

To compute it, we need to perform the following operations:



2.1 CPU: What does it look like?

Core 0
(ALU, branch predictor,
OoO scheduler)

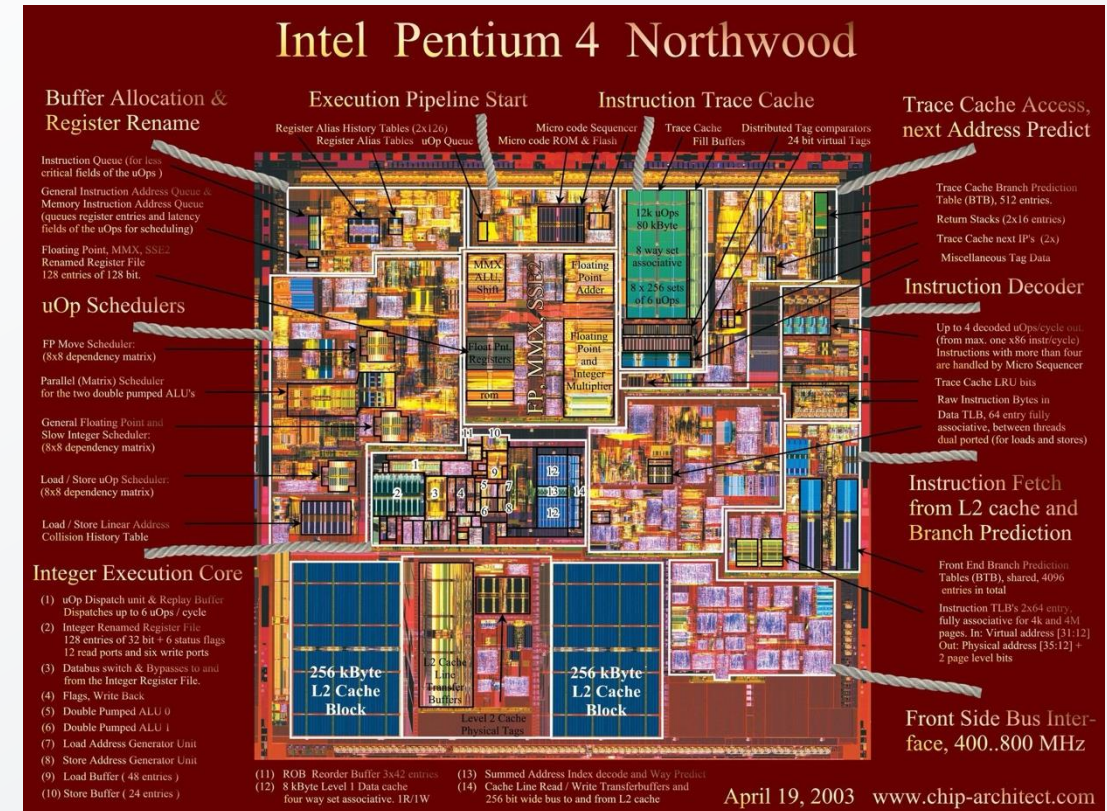
Core 1

Core 2

Core 3

Shared L2 / L3 cache

Memory controller & I/O



<https://superuser.com/questions/324284/what-is-meant-by-the-terms-cpu-core-die-and-package>

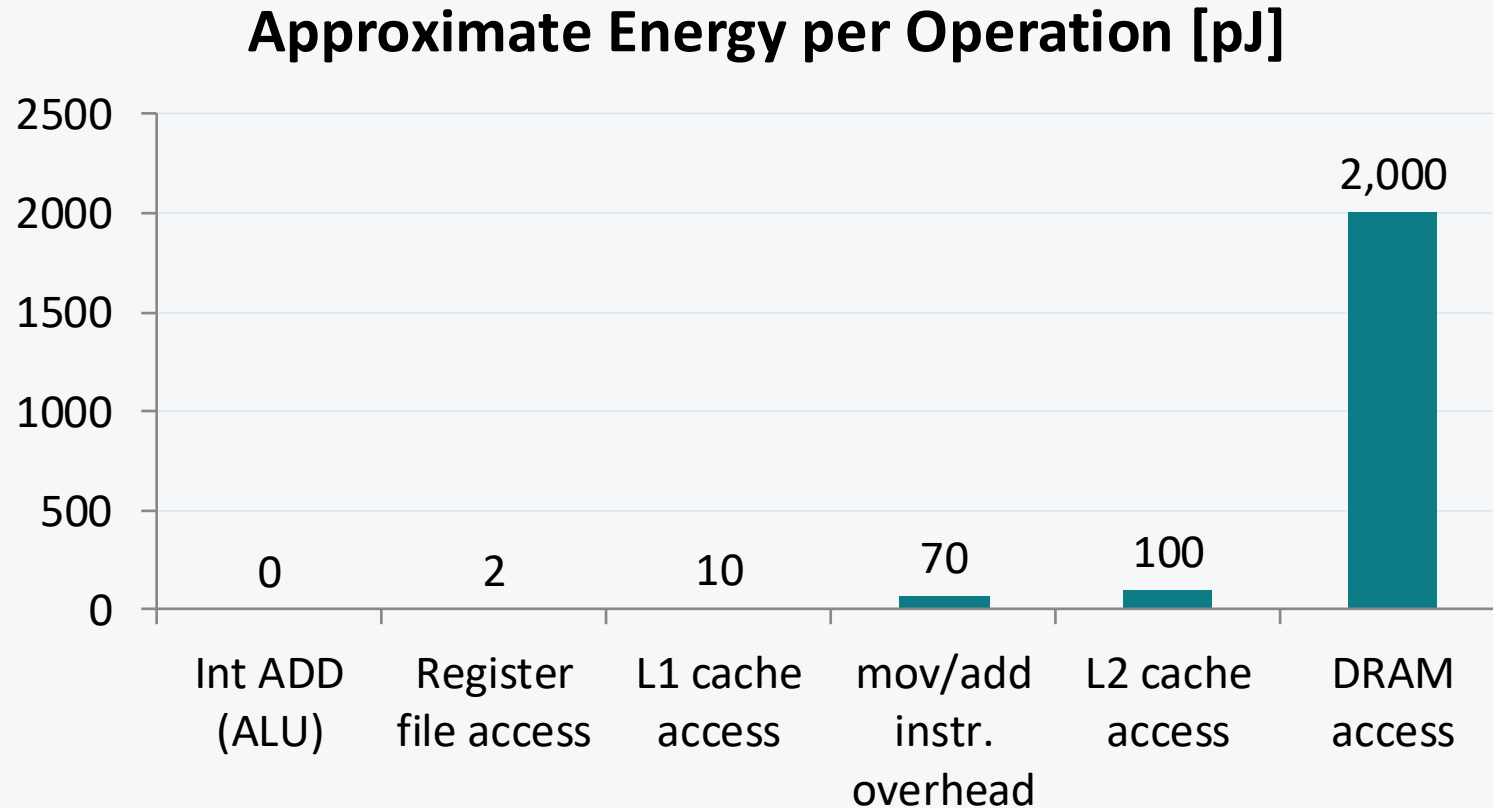
2.2 CPU: Energy cost of instructions

Same 5 stages, now with their energy costs (45nm-class CPU, Horowitz 2014):

Fetch	Read instruction from L1-I cache	~10 pJ
Decode	Translate opcode into micro-ops	*part of ~70 pJ overhead
Execute	ALU executes the arithmetic operation	~0.1 pJ (tiny!)
Memory Access	Access data cache – or DRAM on a miss	10 pJ (cache hit) → 2000 pJ (DRAM)
Write-back	Save result to register file	~1-3 pJ

*Overhead of fetching/decoding/scheduling an instruction usually outweighs the actual arithmetic cost.

2.3 CPU: Operation's cost comparison



Why DRAM dominates

- A single DRAM load costs >10x than all the instructions in a small addition combined
- PUSH and CALL both touch memory (the stack), so their cost depends heavily on whether that memory is cached

2.4 CPU: Energy cost, instruction by instruction

$$y = \text{ReLU}(x_1w_1+x_2w_2+x_3w_3+x_4w_4+b)$$

Load operands (8×)

movss	xmm0,	[x1]	80	pJ
movss	xmm1,	[w1]	80	pJ
movss	xmm2,	[x2]	80	pJ
movss	xmm3,	[w2]	80	pJ
movss	xmm4,	[x3]	80	pJ
movss	xmm5,	[w3]	80	pJ
movss	xmm6,	[x4]	80	pJ
movss	xmm7,	[w4]	80	pJ

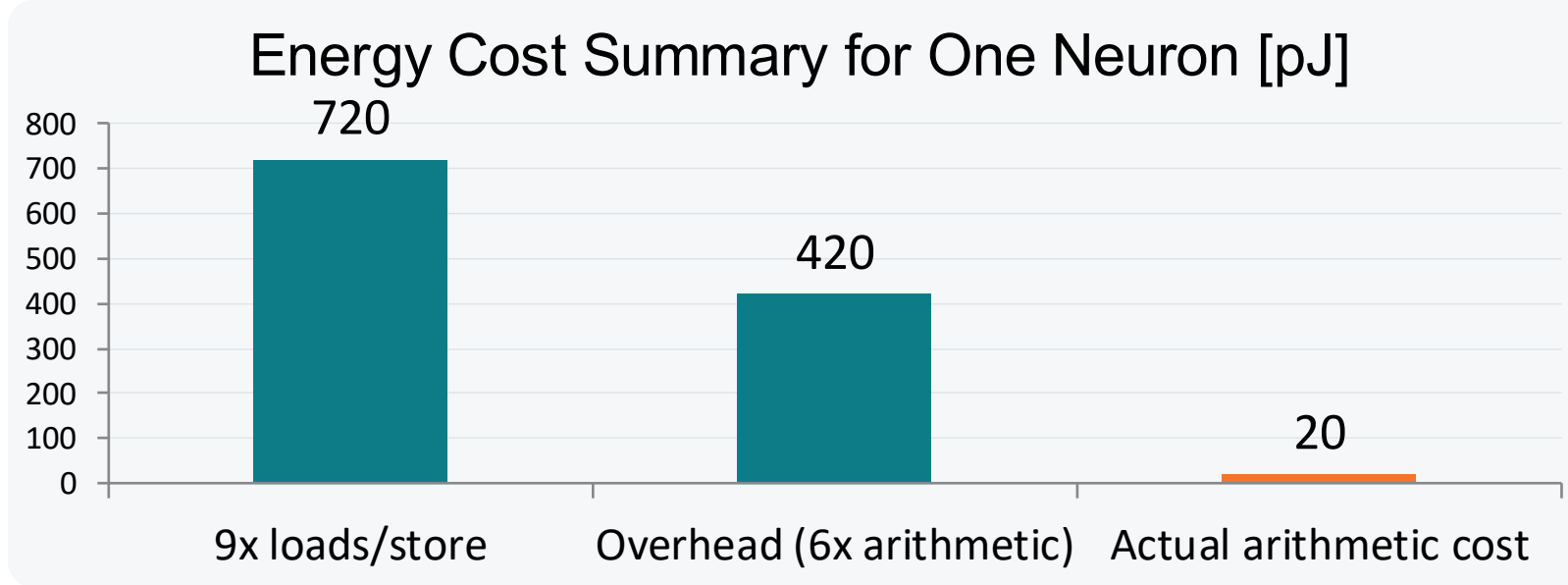
Multiply-accumulate, bias, ReLU, store (7×)

fmadd	acc,	xmm0,	xmm1	75	pJ
fmadd	acc,	xmm2,	xmm3	75	pJ
fmadd	acc,	xmm4,	xmm5	75	pJ
fmadd	acc,	xmm6,	xmm7	75	pJ
add	acc,	bias		70	pJ
max	acc,	0		70	pJ
movss	[y],	acc		80	pJ

8 loads + 4 FMAs + 1 bias add + 1 ReLU compare + 1 store =
15 separate instructions, 1,160 pJ total

Horowitz, M. (2014). 1.1 Computing's energy problem (and what we can do about it). In 2014 IEEE International Solid-State Circuits Conference (ISSCC) (pp. 10-14).

2.5 CPU: RELU energy cost summarized



Why instruction overhead dominates

Of the 6 arithmetic instructions' 440 pJ:

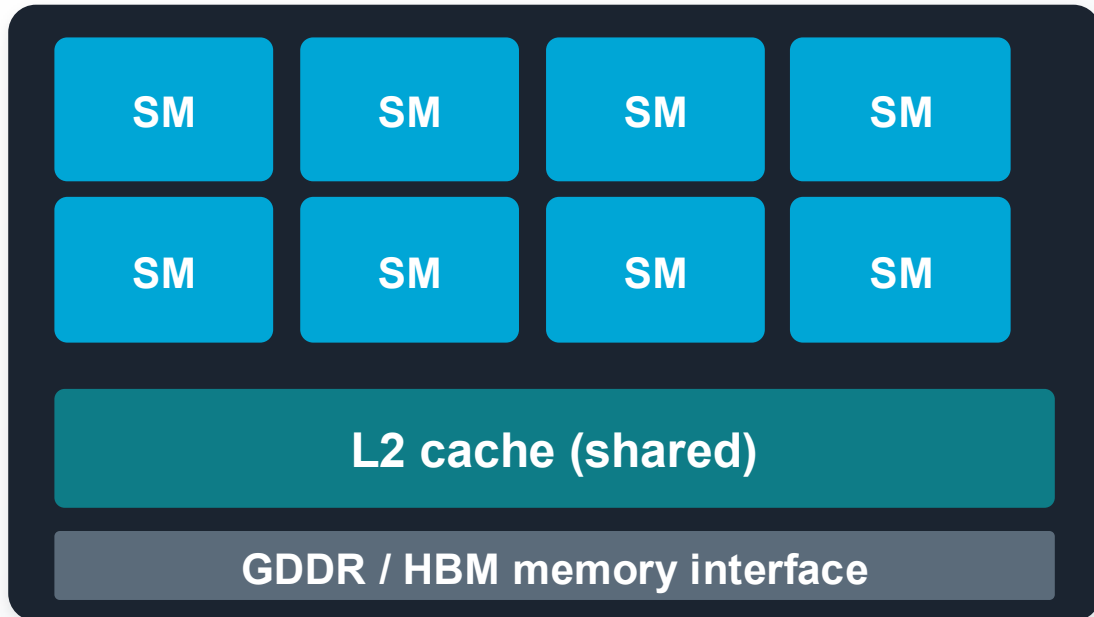
~420 pJ is the ~70 pJ per instruction fetch/decode overhead

PART 2 — INSIDE THE HARDWARE

How does it look like on a GPU?

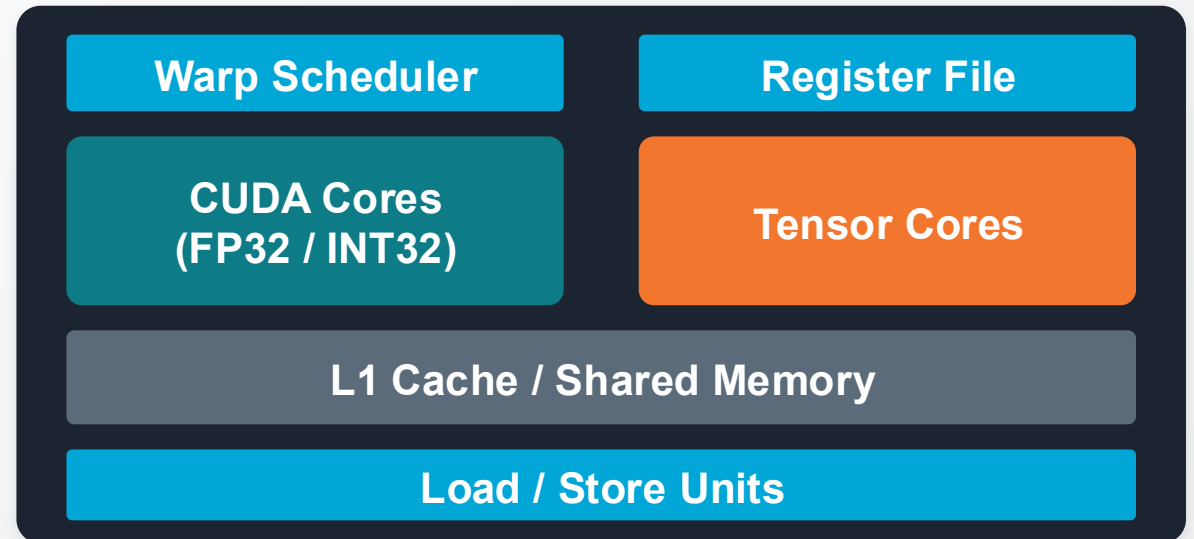
3.1 GPU: what is a Streaming Multiprocessor (SM)?

Every GPU is physically built from many identical, independent processing units called Streaming Multiprocessors (SMs):



Many small streaming multiprocessors (SMs), not a few large cores like a CPU.

Inside one SM (simplified)



A real SM (e.g. NVIDIA's Ada/AD102) has more sub-blocks and varies by generation.

Source: TechPowerUp, "NVIDIA Ada AD102 Block Diagram and New Architectural Features Detailed" [techpowerup.com/299130](https://www.techpowerup.com/299130)

3.2 GPU: threads, warps & thread blocks

The hierarchy that makes SIMT work: threads are grouped into warps, warps into thread blocks, thread blocks into a grid.

GRID — every thread block launched for one kernel call

THREAD BLOCK — up to 1024 threads, share one pool of on-chip SRAM

WARP — 32 threads

THREAD

1 lane, 1 data element, no control logic of its own

CPU thread ≠ GPU thread

A CPU thread is an independent instruction stream with its own control logic. A GPU thread is one lane in a 32-wide lockstep group. The warp scheduler controls all 32 at once.

3.3 GPU: SIMT — one instruction, many lanes

Every thread in a warp executes the exact same instruction, each on its own data. This is what SIMT (Single Instruction, Multiple Threads) means in practice.

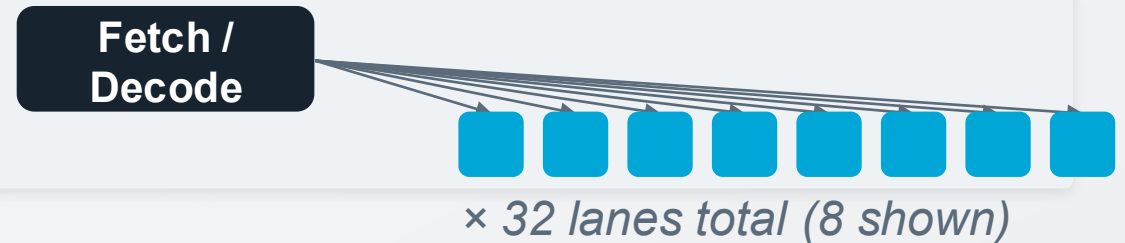
CPU fetch/decode

1 instruction fetched → 1 ALU op executed



GPU SIMT fetch/decode

1 instruction fetched → up to 32 ALU ops executed, same cycle



3.4 GPU: Memory hierarchy & hiding latency

Global memory (DRAM)

Off-chip, big, slow, energy-expensive. Where kernel inputs/outputs are.

Shared memory (on-chip SRAM)

On-chip, shared within a thread block. Much cheaper per access than DRAM.

Hiding memory latency

Warp A **RUNNING**

Warp B waiting on DRAM

When A stalls, the scheduler instantly swaps in B.

Why a Streaming Multiprocessor (SM) keeps many warps resident at once

A warp stalling on a DRAM read (hundreds of cycles) would leave its ALUs idle.

The SM keeps dozens of warps resident at once and switches between them the instant one stalls: throughput-oriented latency hiding, the opposite of a CPU's large caches and branch prediction, which try to avoid the stall happening in the first place.

3.5 GPU: Energy cost, thread by thread

Illustrative PTX-style pseudocode for the same 8-thread neuron kernel:

```
// 8 threads: one per (x_i, w_i) pair
ld.global.f32 %v, [buf + tid*4] // Load x_i/w_i from          5,120 pJ (8x)
DRAM
mul.f32 %p, %v, %w[tid] // x_i * w_i                        40 pJ (8x)
st.shared.f32 smem[tid], %p // stash product in shared
mem

// thread 0 only: reduce, bias, ReLU
ld.shared.f32 %s, smem[0..7] // read back all 8              40 pJ (8x)
products
add.f32 %acc, %s0..%s7 // accumulate the sum
add.f32 %acc, %acc, %bias // add bias b                      ≈20 pJ compute
max.f32 %acc, %acc, 0.0 // ReLU
st.global.f32 [y], %acc // write y to DRAM                  640 pJ
```

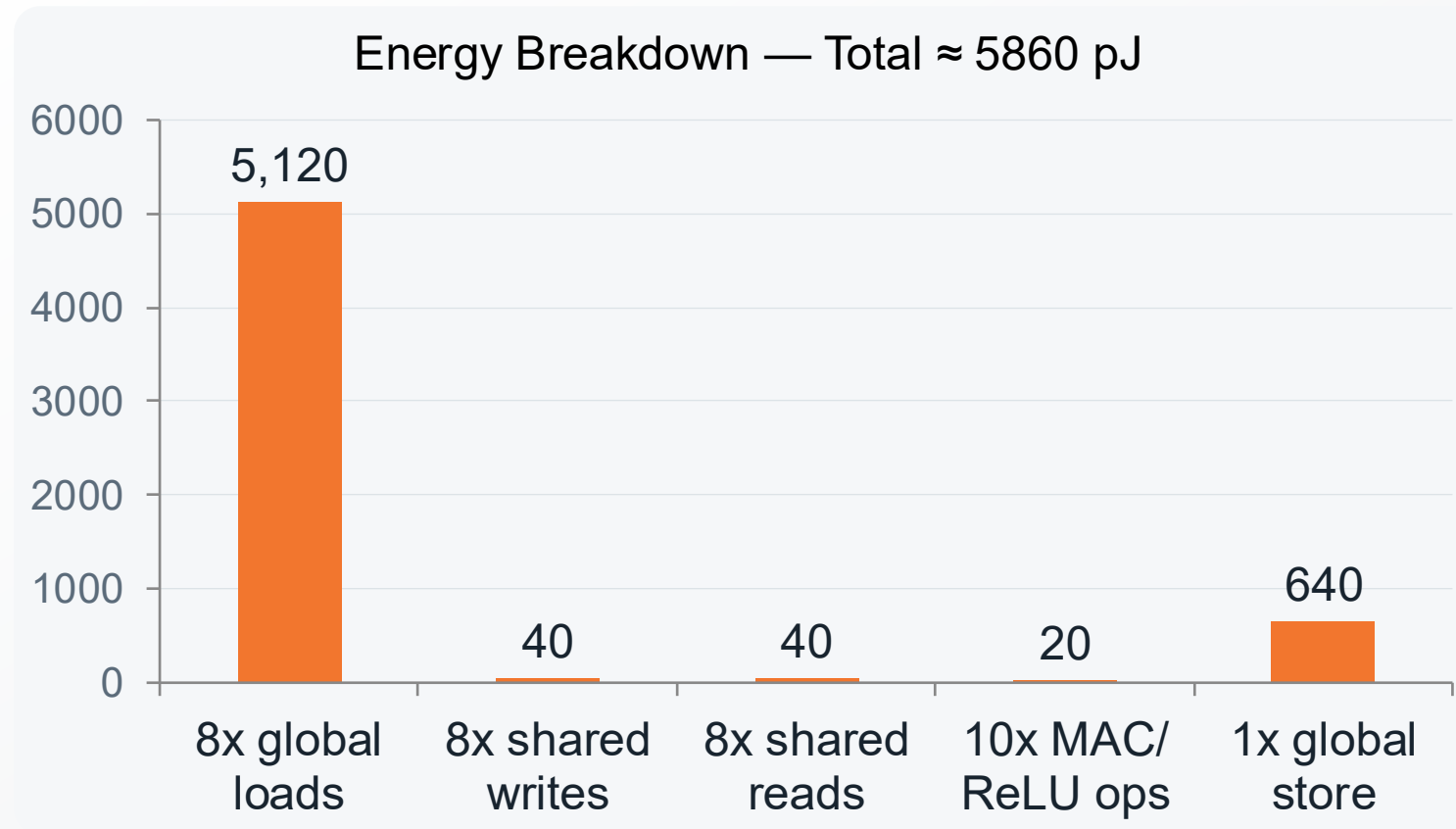
No ~70 pJ-per-thread fetch cost here: one instruction dispatch covers the whole warp.

Instead, the cost is entirely in the DRAM round trips.

Dally, W. (2015). High-performance hardware for machine learning [NIPS 2015 Tutorial]. Reproduced in Stanford CS231n (2017).
Horowitz, M. (2014). 1.1 Computing's energy problem (and what we can do about it). In 2014 IEEE ISSCC (pp. 10-14).

3.6 GPU: Energy comparison

Same neuron, 8 threads: each loads one input or weight from global memory, multiplies, writes to shared memory, then thread 0 accumulates, adds the bias, applies ReLU, and stores the result:



Worse, not better

- Compute (4 FMA + bias + ReLU): ~ 20 pJ
- On-chip shared accesses: ~ 80 pJ
- Off-chip DRAM traffic: $\sim 5,760$ pJ ($\sim 98\%$)

One isolated neuron can't fill a GPU's 32-wide warp. The fixed cost of reaching DRAM 8 separate times outweighs everything. GPUs win once there are thousands of neurons to run at once.

PART 2 — INSIDE THE HARDWARE

More specialized chips, same set of physical constraints

4.1 TPU: built around one operation, matmul

A Tensor Processing Unit is designed around tensor contractions, especially matrix multiplication, because modern ML workloads reduce to repeated matmul-like ops.

$$Y = X \cdot W + b$$

Why matmul is (almost) all a TPU needs to be fast at

- Dense/linear layers: $y = Wx + b$ is a matmul.
- Attention: $Q \cdot K^T$, then $\cdot V$, are two chained matmuls.
- Convolution: unrolled (im2col) into a matmul.

What does a TPU look like inside?

**Systolic MAC array (e.g.
256×256 PEs)**

**Unified buffer
(SRAM)**

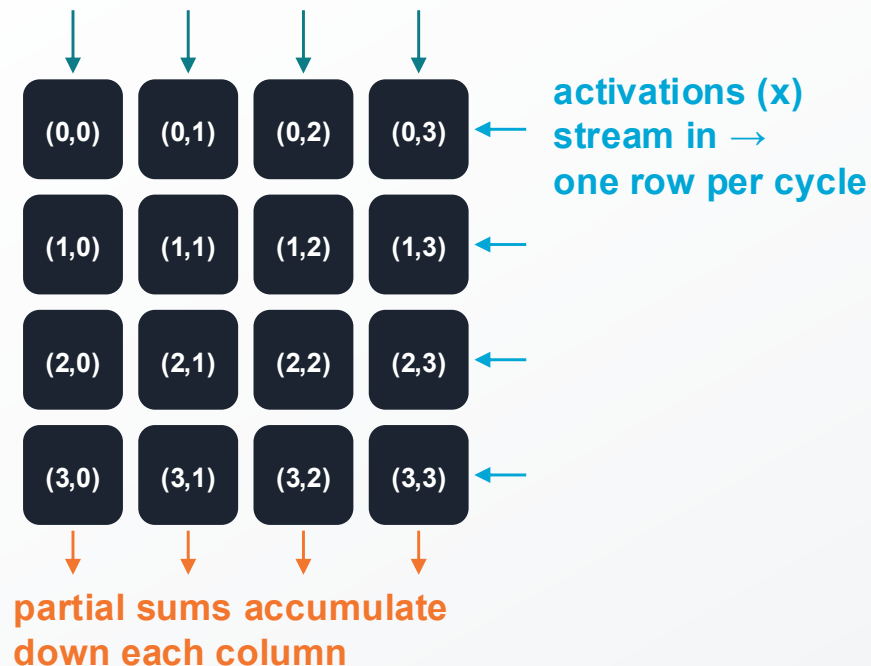
Control & host I/O

Why the array dominates the die

- Almost no branch prediction, speculation, or out-of-order logic: matmul has no branches to predict.
- Most transistors go straight into the grid of multiply-accumulate cells itself.

4.2 TPU: how a systolic array actually computes it

weights (w) loaded once,
held resident ↓



Each PE does one multiply-add per cycle and passes its result to its neighbor: a wave of computation flows across the array without a single extra fetch.

Two ways to describe the same matmul:

```
%y = "mhlo.dot_general"(  
    %x, %w, %b)  
    : (tensor<1x4xf32>,  
      tensor<4x1xf32>,  
      tensor<1xf32>)  
    -> tensor<1xf32>
```

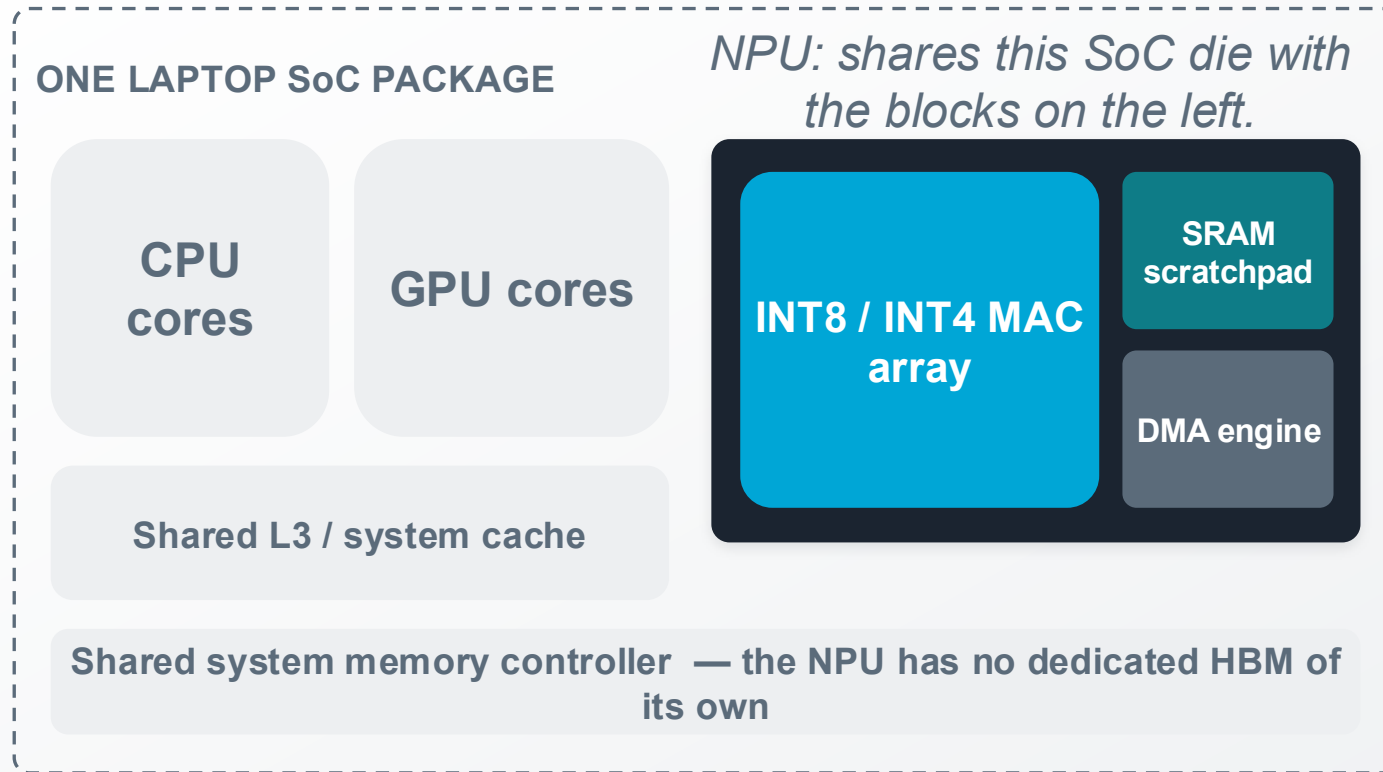
① What the compiler sees: one high-level op: no separate loads, multiplies, or adds.

```
LOAD    w[0..3] -> PE[0..3]  
STREAM  x[0..3] -> array  
MAC     x[i]*w[i], all 4  
DRAIN   sum -> +bias -> y
```

② What the array executes: one dispatch drives all 4 MACs. The CPU needed ~15 instructions!!

4.3 NPU: what does an NPU look like inside?

An NPU is usually a small block sharing the same die, and the same power budget, as the CPU and GPU cores.

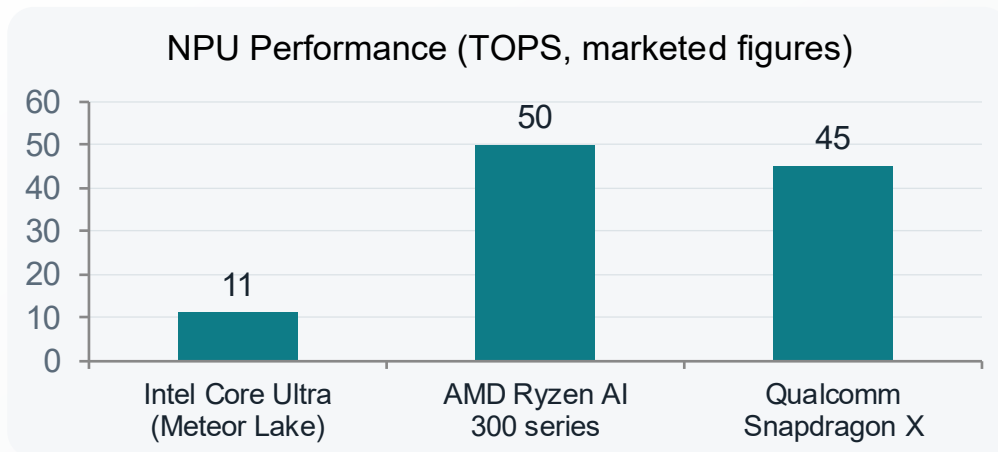


How an NPU works

- A small array of INT8/INT4 multiply-accumulate units. Far fewer than a GPU's, but purpose-built for exactly this operation.
- A local SRAM scratchpad keeps weights/activations close to the MACs; a DMA engine streams data in from shared system memory.
- Fixed-function, not general-purpose: it runs a narrow set of NN operators well instead of arbitrary code.
- Tuned for low-batch, single-inference latency at a few watts.

4.4 NPU: the always-there AI engine on your laptop

A Neural Processing Unit is the on-device, power-efficient accelerator for matmul/conv kernels.



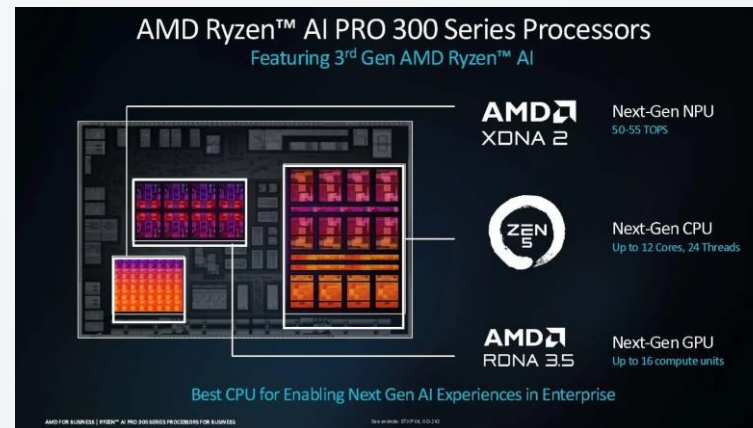
What are these three, and are they all NPUs?

Yes. Each name is a laptop System-on-Chip (SoC) that bundles a CPU, a GPU and an NPU on one die. The bar shows only that chip's NPU block. Values differ because the NPUs differ: different architecture and generation, different process nodes, and figures measured at different precision.

Why NPUs suddenly matter

Local LLM inference needs sustained low-power AI compute that CPUs and GPUs alone aren't efficient enough to sustain always-on.

Example package: AMD Ryzen AI 300 (“Strix Point”)



One die, three engines.

The NPU (XDNA 2) is a small fraction of the die. Most of the area is still CPU and GPU.

4.5 Our neuron on a TPU and an NPU

Both run the same multiply-accumulate-bias-ReLU pattern, but a single neuron can't show their real advantage.

TPU: ~0.4–0.5 pJ per MAC (system-level, at full array utilization)

Derived from published chip specs:

Edge TPU = 4 TOPS at 2 W (Google) → 0.5 pJ/op;

Cloud TPU v1 = 92 TOPS at 40 W (Jouppi et al., 2017) → 0.43 pJ/op.

Both assume thousands of MACs running in parallel. A single neuron can't reach that.

Sources: coral.ai/docs/edgetpu/faq/ · Jouppi et al., "In-Datacenter Performance Analysis of a TPU," ISCA 2017, arxiv.org/abs/1704.04760

NPU: no honest pJ number exists to give

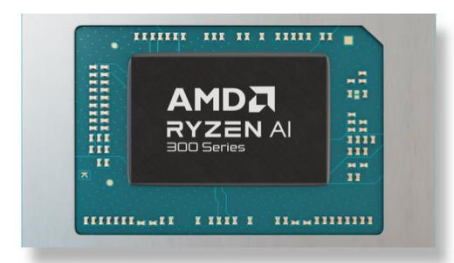
AMD, Qualcomm and Intel publish whole-SoC TOPS and whole-SoC TDP, never the NPU block's own power draw. It shares one die and one power budget with the CPU and GPU cores.

Checked, not found: AMD Ryzen AI / Qualcomm Snapdragon X product briefs (whole-SoC TOPS & TDP only, no NPU-isolated figure published)

Why they're efficient anyway

- Weights are reused across thousands of MACs (weight-stationary reuse) instead of round-tripping to DRAM per neuron
- One instruction drives a whole array of MAC units, so the ~70 pJ CPU-style fetch/decode cost is paid once, not per operation

5 Chips comparison (2025-26)

Class	Picture	Example	Max Power Draw	Memory	Node
CPU		Intel Core Ultra 7 155H	115 W max (28 W base)	System RAM + cache	Multi-tile (Intel+TSMC)
GPU		NVIDIA RTX 4090	450 W max (board power)	24GB GDDR6X	TSMC 4N
TPU (edge)		Google Coral Edge TPU	2 W max (Edge TPU)	On-chip SRAM + host RAM	Varies
NPU		AMD Ryzen AI 300	SoC TDP (NPU itself: not disclosed)	Unified system memory	SoC (vendor)
NPU		Qualcomm Snapdragon X	SoC TDP (NPU itself: not disclosed)	Unified memory	SoC (vendor)

TDP = Thermal Design Power, the heat/power figure the cooling system is built for. **SoC** = System-on-Chip (CPU+GPU+NPU on one die).

Node = manufacturing process (smaller = newer/denser).

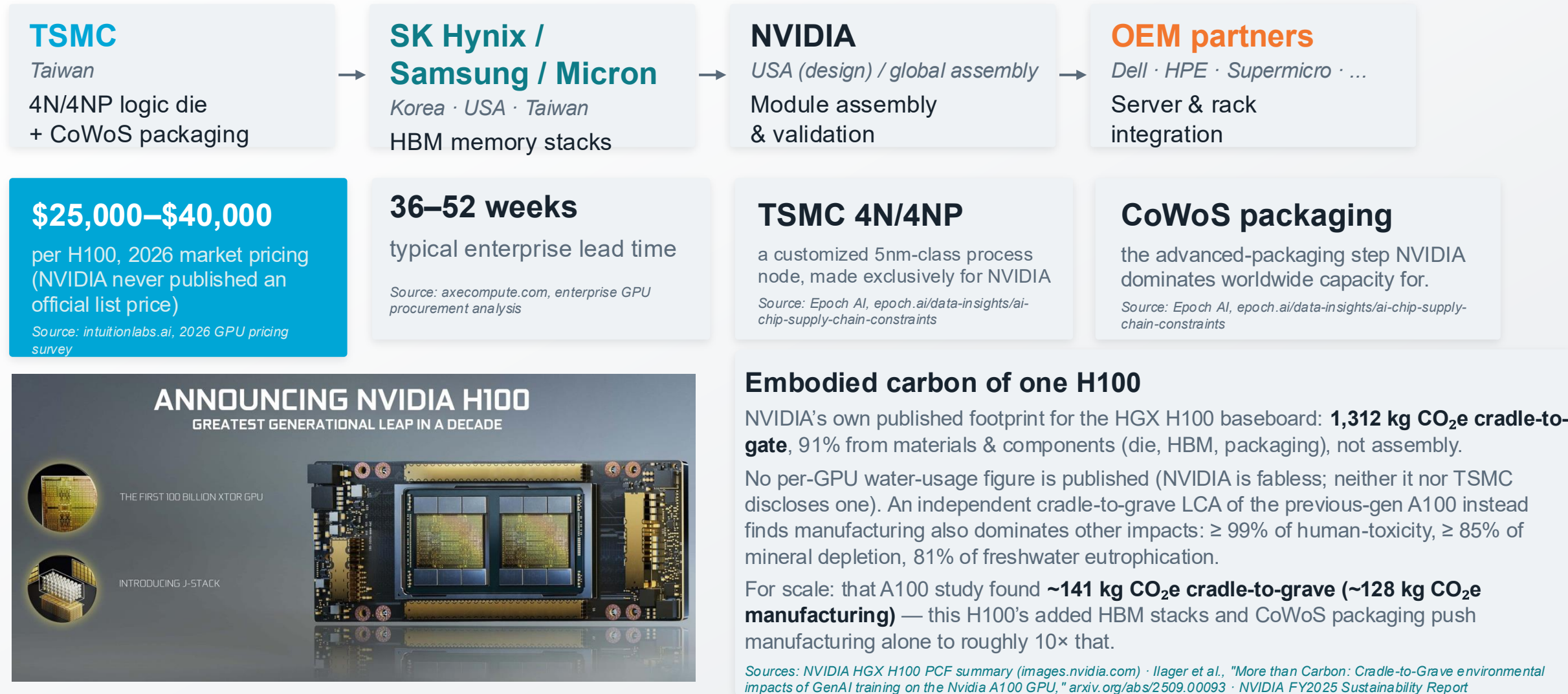
GDDR6X / HBM / SRAM = memory types (off-chip graphics memory, stacked high-bandwidth memory, on-chip fast memory).

Transistor counts for laptop SoCs/NPUs are often undisclosed.

Sources: ark.intel.com (Core Ultra 7 155H) · nvidia.com/geforce/rtx-4090 · coral.ai/docs/edgetpu/faq · amd.com (Ryzen AI 300 launch, Oct 2024) · qualcomm.com (Snapdragon X Elite product brief)

6.1: The anatomy of an NVIDIA GPU

Following one accelerator, **an NVIDIA H100**:



6.2: From one GPU to a datacenter

An example of “a typical data center”: xAI's Colossus cluster in Memphis:

230,000 GPUs

150,000 H100 + 50,000
H200 + 30,000 GB200

~250 MW

35 gas turbines + Tesla
Megapacks power the site

1,500 racks

64 liquid-cooled GPUs per
rack

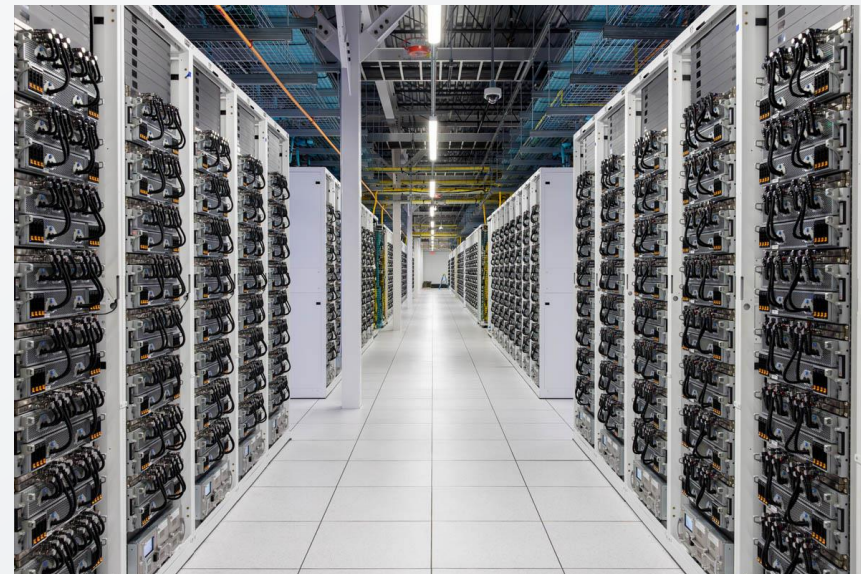
122 + 92 days

to build the first 100,000 GPUs,
then the next 100,000+

Every one of those 230,000 GPUs is still just running the same four-input neuron, multiplied by billions.



<https://www.teslarati.com/elon-musk-xai-brings-1gw-colossus-2-ai-training-cluster-online/>



<https://www.servethehome.com/inside-100000-nvidia-gpu-xai-colossus-cluster-supermicro-helped-build-for-elon-musk/>

7 Same constraints, four different approaches

CPU

- Optimizes single-thread latency
- ILP within one core
- Deep caches hide DRAM stalls
- Best for: branchy, irregular code

GPU

- Optimizes many-thread throughput
- TLP across warps
- Bandwidth + shared-memory reuse
- Best for: regular, data-parallel math

TPU

- Optimizes matmul throughput
- One instruction drives thousands of MACs
- Weight-stationary reuse
- Best for: large, batched linear algebra

NPU

- Optimizes TOPS-per-watt at low power
- Same idea as TPU, tuned for always-on edge use
- Shares SoC die & power budget
- Best for: sustained on-device AI (Copilot+, local LLMs)

Exam(ple) questions

1. Which statement about a GPU's Streaming Multiprocessors (SMs) is false?

- a. A modern GPU is physically built from many small, identical SMs, not a few large cores like a CPU.
- b. Each SM has its own warp scheduler, register file, and CUDA/tensor cores.
- c. A single thread block is split across multiple SMs at once, with each SM running part of it.
- d. A warp, 32 threads executing one shared instruction per cycle, in lockstep, is the real unit the hardware schedules.

2. Which statement about the CPU's per-instruction energy cost (as traced through the four-input neuron example) is false?

- a. Fetching/decoding/dispatching an instruction costs roughly the same ~70 pJ overhead whether it's a load or an arithmetic op.
- b. Most of the 1,160 pJ needed to compute one neuron on a CPU goes to the actual multiply-add arithmetic, not to instruction overhead.
- c. A DRAM access on a cache miss costs on the order of 200× more energy than a cache hit.
- d. Computing the neuron takes 15 separate instructions: 8 loads, 4 fused multiply-adds, 1 bias add, 1 ReLU compare, and 1 store.

Exam(ple) questions — Answers

1. Which statement about a GPU's Streaming Multiprocessors (SMs) is false?

- ~~a. A modern GPU is physically built from many small, identical SMs, not a few large cores like a CPU.~~
- ~~b. Each SM has its own warp scheduler, register file, and CUDA/tensor cores.~~
- c. A single thread block is split across multiple SMs at once, with each SM running part of it.**
- ~~d. A warp, 32 threads executing one shared instruction per cycle, in lockstep, is the real unit the hardware schedules.~~

2. Which statement about the CPU's per-instruction energy cost (as traced through the four-input neuron example) is false?

- ~~a. Fetching/decoding/dispatching an instruction costs roughly the same ~70 pJ overhead whether it's a load or an arithmetic op.~~
- b. Most of the 1,160 pJ needed to compute one neuron on a CPU goes to the actual multiply-add arithmetic, not to instruction overhead.**
- ~~c. A DRAM access on a cache miss costs on the order of 200× more energy than a cache hit.~~
- ~~d. Computing the neuron takes 15 separate instructions: 8 loads, 4 fused multiply-adds, 1 bias add, 1 ReLU compare, and 1 store.~~

Recap

One instruction, four chips

The same multiply-add was traced through a CPU, a GPU, a TPU and an NPU.

Parallelism has a price

A GPU runs threads in lockstep (SIMT). It is only efficient when those threads agree - divergence and memory stalls set its real energy cost.

Specialization wins

TPUs and NPUs trade generality for far better performance-per-watt on matmul-heavy AI workloads.

Instruction overhead adds up

Fetch, decode and control logic cost energy before any real math happens; that overhead is exactly what specialised chips are built to remove.

Difference between CPU and GPU

CPU optimizes for single-thread latency; GPU optimizes for many-thread throughput

The chip itself has a cost

Manufacturing: silicon, advanced packaging, rare materials, is a real, often invisible part of a chip's footprint, before it ever computes anything.

Sustainability recommendation: these accelerators are indispensable, but typically ship with only a 3–4 year warranty and little to no technical support after that – and are largely decommissioned, not upgraded, once that window closes. Plan hardware lifecycles around that reality, not around theoretical silicon lifespan.

Lecture Feedback

